

文件系统：名称、对象、索引与持久化

从路径、fd、inode、块映射到页缓存、日志与崩溃一致性

408 专题手册 · 操作系统文件系统篇

核心模型：文件系统建立持久对象世界

文件系统在块式持久存储之上，为程序制造一个看起来拥有**名字、对象、层次、共享、保护、随机访问与长期存在**的世界。

分析主线：

Name → Object → Open State → Byte Range
→ Cached Page → Block Mapping → Persistent State

横向始终检查三个约束：

Protection

Lifecycle

Crash Consistency

学生版一句话：先找到“谁”，再建立访问关系，再找到“哪一段数据”，最后保证它即使掉电也仍然是一个合法文件。

术语预备：名字、对象与持久数据

- **文件对象**：可按字节读写并具有生命周期的逻辑对象；**文件名**只是目录中指向它的一条名称记录，不是对象身份本身。
- **元数据 (metadata)**：描述对象类型、大小、权限、时间、所有者和数据位置的信息；它与文件内容分开管理。
- **目录项 (directory entry)**：目录文件中的“名字 o 对象标识”记录；**dentry** 是 Linux VFS 在内存中缓存路径组件的对象，不能直接当成磁盘目录项。
- **FCB/inode**：FCB 是教材中的文件控制块；inode 是 Unix 类系统保存文件身份、元数据和数据定位信息的结构。两者都不是文件名本身。
- **文件描述符 (fd)**：进程文件描述符表中的小整数索引；**OFD** 是它指向的打开文件对象，保存当前偏移和打开方式；**fd**、**OFD**、**inode** 是三层不同对象。
- **ACL**：Access Control List，按对象记录哪些主体拥有何种权限；**Capability**：按主体持有可访问对象及权限的凭证集合。
- **VFS**：Virtual File System，向应用提供统一的文件 API，并把路径操作转发给具体文件系统实现；**mount** 把一个文件系统接入现有目录命名空间。
- **页缓存 (Page Cache)**：以内存页为单位缓存文件内容，供普通读写和 **mmap** 共享；它不是 CPU 的硬件 Cache。
- **日志/预写日志 (Journaling/WAL)**：先把元数据变更意图按规定顺序写入日志，崩溃恢复时据此重放或撤销，保证文件系统结构不落入非法中间状态。

后文遇到路径、**open**、**fork**、硬链接或 **mmap** 时，都沿着“名字 o 对象 o 打开引用 o 数据位置 o 持久化约束”这条链判断。

系统的通用推理：崩溃一致性 (Crash Consistency) 中的元数据不变量

在文件系统持久化与崩溃恢复中，同样遵循 OS 通用系统推理引擎 (OS System Reasoning Engine)：

State (Bitmap/Inode/Data Blocks) → Crash/Power Fault (突发断电)
 → Failure (块被重复分配/孤立)
 → Invariant (Bitmap Free $\iff$ 无 Inode 占用)

为了拦截这类毁坏性坏状态，文件系统引入了**预写日志 (Journaling / WAL)** 或**写顺序约束 (Write-Ordering)** 作为最小机制，确保磁盘元数据在故障恢复后依然满足逻辑一致性。

目录

1 第 0 章：文件系统究竟解决什么问题	3
1.1 从裸块设备到“文件世界”	3
1.2 五层映射 + 一个持久化约束	3

2 Part I：名字怎样找到对象	4
3 第 1 章：文件抽象、元数据与对象身份	4
3.1 文件不是“磁盘上一段连续字节”	4
3.2 FCB、inode 与“文件名在哪”	4
3.3 文件逻辑结构、访问方式与物理分配的三层辨析	5
3.4 文件保护模型：Access Matrix 与 ACL / Capability 投影	5
4 第 2 章：目录、路径与链接——从名字到对象	6
4.1 目录：把文件名映射到文件对象	6
4.2 路径解析不是“一次查表”，而是状态推进	6
4.3 绝对路径、相对路径与搜索起点	6
4.4 硬链接：多个名字，同一个对象	7
4.5 符号链接：一个独立对象，内容解释为路径	7
5 第 3 章：VFS 与 Mount——把多个文件系统拼成一个命名空间	7
5.1 为什么需要 VFS	7
5.2 VFS 四类核心对象的职责	8
5.3 Mount：路径经过挂载点时切换文件系统实例	8
6 Part II：进程怎样引用已经找到的文件	8
7 第 4 章：fd、打开文件描述与 inode	8
7.1 为什么不能让 fd 直接等于 inode	8
7.2 三种最重要的共享关系	9
7.3 一个稳定的题目判断法	9
8 第 5 章：文件生命周期——名字引用与打开引用是两套生命线	9
8.1 unlink 删除的首先是一条命名边	9
8.2 双生命周期模型	10
8.3 create、link、unlink、rename 分别改什么	10
9 第 6 章：文件保护——谁可以对哪个对象做什么	10
10 Part III：文件中的字节怎样落到块上	11
11 第 7 章：从文件偏移到逻辑块	11
12 第 8 章：文件物理结构——三种经典方案从矛盾中产生	11
12.1 连续分配：数组模型	11

12.2 链接分配：链表模型	11
12.3 索引分配：映射表模型	11
13 第 9 章：inode 多级索引——最大文件与 I/O 次数计算	12
13.1 经典 UNIX-style mixed indexing	12
13.2 I/O 次数一定先写缓存假设	12
14 Part IV：空间从哪里来——Global Layout 与 Free Space	13
15 第 10 章：文件系统全局 Layout	13
15.1 为什么必须有全局账本	13
16 第 11 章：外存空闲空间管理	14
16.1 位图 Bitmap	14
16.2 空闲链表 Free List	14
16.3 成组链接 Grouped Linking	14
16.4 计数法 Counting	14
17 Part V：文件操作怎样改变内核状态	14
18 第 12 章：一次 open() 的一生	14
18.1 open 的高层语义	14
19 第 13 章：一次 read(fd, buf, n) 的文件系统路径	15
19.1 先定位打开实例，再定位文件位置	15
19.2 现代 buffered read：先问页缓存	15
19.3 跨块读取	15
20 第 14 章：一次 write(fd, buf, n) 的文件系统路径	16
20.1 Buffered write 的核心状态变化	16
20.2 写入可能同时修改哪些状态	16
21 第 15 章：create、close、unlink、rename 的状态机	16
21.1 create：先创造身份，再绑定名字	16
21.2 close：释放一个 handle，不等于删除文件	17
21.3 rename：名字变化不等于内容复制	17
22 Part VI：为什么“写完”以后文件系统仍可能坏	17
23 第 16 章：Crash Consistency——持久状态机的核心问题	17
23.1 高层一次操作，底层多次写	17

23.2 崩溃分析五步法	17
24 第 17 章：fsck 与 Journaling	18
24.1 fsck：允许不一致，重启后扫描修复	18
24.2 Journaling：先记录一笔可重放的事务	18
24.3 Consistency 与 Durability 不完全是一件事	18
25 第 18 章：概念边界	18
26 第 19 章：最小调用接口	19
27 附录 A：考试模型 / 机制 / 工程边界	20
28 附录 B：参考资料与工程边界	20

1 第 0 章：文件系统究竟解决什么问题

1.1 从裸块设备到“文件世界”

假设底层只给你一串可以读写的编号块：

$$B_0, B_1, B_2, \dots, B_{N-1}.$$

单靠这些块，应用无法自然回答：

- 这堆字节叫什么名字？目录层次在哪里？
- 这段数据属于哪个逻辑对象？对象有多大、谁拥有、谁能读写？
- 文件增长以后，新块从哪里来？删除以后空间怎样回收？
- 多个名字、多个进程能否指向同一对象？
- 内存缓存和磁盘状态不同步时，什么叫“写成功”？
- 创建文件需要改多个磁盘结构，掉电发生在中间怎么办？

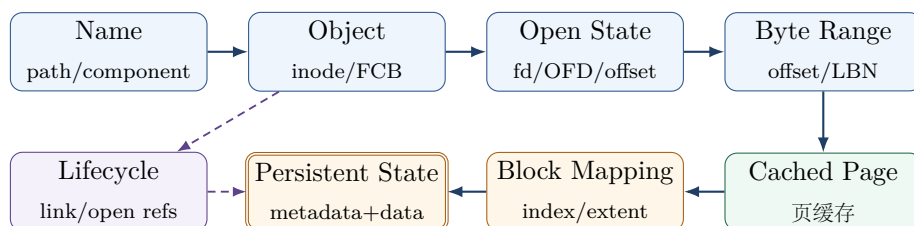
因此文件系统至少提供四个承诺：

承诺	对应用提供的幻觉/抽象	内核必须解决的问题
Naming	用路径而不是块号找对象	目录、路径解析、mount、链接
Object	把字节组织成“文件”	inode/FCB、元数据、权限
Access	打开后可持续读写	fd、打开文件描述（OFD）、offset
Persistence	重启后对象仍可解释	layout、分配、写回、日志、恢复

文件系统与前面专题的边界

- **I/O 专题**回答：已经知道要访问哪个块以后，请求如何提交、DMA 如何搬运、设备如何完成、进程如何被唤醒。
- **虚拟内存专题**回答：虚拟页怎样映射到物理页；`mmap()` 时文件页怎样成为虚拟地址的后备对象。
- **文件系统专题**回答：`/a/b/c` 或 `fd=3` 怎样变成文件对象、文件偏移与存储位置，以及这些持久元数据怎样保持一致。

1.2 五层映射 + 一个持久化约束



文件系统六问

以后任何文件系统机制，都先问：

1. 名字怎样找到对象？
2. 进程怎样在打开以后持续引用对象？
3. 文件中的字节位置怎样找到逻辑块/页？
4. 逻辑块怎样找到持久存储位置？
5. 新空间怎样分配、旧空间怎样回收？
6. 高层操作由多次持久写组成时，怎样抵抗崩溃？

2 Part I: 名字怎样找到对象

3 第 1 章：文件抽象、元数据与对象身份

3.1 文件不是“磁盘上一段连续字节”

从应用角度，普通文件最稳定的抽象通常是一个可按字节定位的持久对象。它同时拥有：

- 内容 (data)；
- 大小、权限、所有者、时间戳等元数据 (metadata)；
- 在特定文件系统对象中的对象身份；
- 将逻辑位置映射到存储空间的方法。

元数据与数据分离

UNIX/inode 思想的关键不是“inode 这个单词”，而是把对象身份与元数据从名字和数据内容中拆开：

$$\text{Name} \neq \text{Object} \neq \text{Data}$$

文件名属于命名空间；inode/FCB 描述对象；数据块保存内容。正因为三者解耦，硬链接、rename、权限修改与共享才有清晰语义。

3.2 FCB、inode 与“文件名在哪”

概念	408 稳定理解	UNIX/Linux 典型映射
FCB	描述文件的控制信息/元数据的抽象	inode 可看作一种具体 FCB 风格实现
inode	对象元数据 + 数据定位信息，不以文件名作为身份	磁盘 inode 与内存 inode object
Directory Entry	把目录中的一个名字映射到对象标识	经典 inode FS 中常是 name → inode number

dentry 不能与“磁盘目录项”混成一个东西

408 中的目录项通常指持久目录文件里的记录；Linux VFS 的 struct dentry 是内存中的路径组件/名称缓存对象。Linux VFS 文档明确说明 dentry 存在于 RAM、用于 pathname lookup，不保存到磁盘。正式做题时先判断题目说的是“目录文件里的 entry”还是“VFS dentry”。

3.3 文件逻辑结构、访问方式与物理分配的三层辨析

408 经常考查文件的不同维度，必须严格区分这三个独立概念：

Logical Structure ≠ Access Method ≠ Physical Allocation

文件系统维度	核心定义与视角	典型实现/分类形式
逻辑结构 (Logical Structure)	** 应用程序 ** 看得到的数据组织形式	无结构流式字节文件、记录式文件（顺序/索引记录）
访问方式 (Access Method)	** 应用程序 ** 从文件中读取/写入数据的接口模式	顺序访问 (Sequential)、直接/随机访问 (Direct/Random)、索引访问
物理分配 (Physical Allocation)	** 操作系统/文件系统 ** 将数据安排落到外存块上的方法	连续分配 (Contiguous)、链接分配 (Linked)、索引分配 (Indexed)

选择文件逻辑结构时看什么

流式文件把内容看成无解释的字节序列；记录式文件再按记录组织。记录式文件中，顺序文件适合按序扫描，索引文件用索引快速定位记录，索引顺序文件兼顾范围检索与顺序处理，散列文件则用 hash 直接定位但不擅长有序遍历。这里讨论的是“应用看到怎样的数据”，不能拿它替换连续/链接/索引分配的外存布局。

3.4 文件保护模型：Access Matrix 与 ACL / Capability 投影

文件的保护与安全机制建立了“谁可以对哪个对象做什么动作”的边界。

通用保护模型：访问矩阵 (Access Matrix) 任何系统保护抽象都可表示为一个二维访问矩阵 AccessMatrix：

AccessMatrix[Subject / Domain, Object] = Rights

其中行代表主体/保护域 (Subject/Domain，如用户、进程、角色)，列代表受保护对象 (Object，如文件、目录、设备)，矩阵元素表示具体的许可权限（如 r, w, x）。

由于系统中的主体和对象极其庞大，矩阵通常非常稀疏。工程中有两种天然的二维存储投影：

投影方式	存储组织与定义	典型实例与特性
按列 (Object) 存储 ACL	→ 将列上的主体及其权限挂在 ** 对象 (File)** 下；形式为：Object → {(Subject _i , Rights _i)}	** 访问控制列表 (Access Control List)**；如 Linux 文件 <code>rwxr-xr--</code> 、posix ACL。删除文件时易一并回收。
按行 (Subject) 存储 Capability	→ 将行上的对象及其权限挂在 ** 主体 (Domain)** 手里；形式为：Subject → {(Object _j , Rights _j)}	** 能力列表 (Capability List)**；进程持有一组令牌/凭证。传递与撤销权限粒度更细，但查询“某文件被谁有权访问”开销高。

ACL 与 Capability 都来自访问矩阵

ACL 与 Capability 根本不是两个孤立无因的名词，它们是 ** 同一个 Access Matrix 的两种存储切割与二维矩阵投影 **！

4 第 2 章：目录、路径与链接——从名字到对象

4.1 目录：把文件名映射到文件对象

在经典 inode 文件系统里，可把目录理解成特殊文件，其内容逻辑上保存：

filename → inode number

因此删除名字与删除对象是两件不同的事；硬链接也不再神秘：它只是**增加一条新的 name → same object 映射**。

4.2 路径解析不是“一次查表”，而是状态推进

解析绝对路径 `/home/user/a.txt` 时，可以抽象为：

$$D_0 \xrightarrow{home} D_1 \xrightarrow{user} D_2 \xrightarrow{a.txt} I.$$

每一步都以“当前目录对象”为状态，以“下一个 pathname component”为输入，执行 lookup 得到下一个对象。

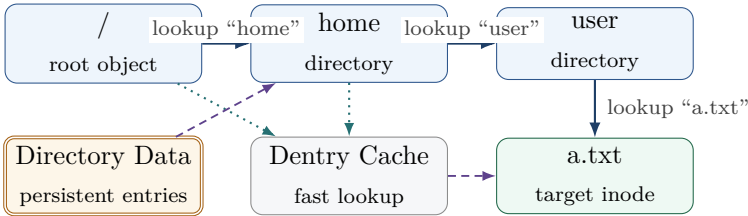


图 1：路径解析：先沿目录逐级推进，dcache 是可选快路径，而不是磁盘目录项本身。

4.3 绝对路径、相对路径与搜索起点

- 绝对路径通常从进程可见的 root 开始；
- 相对路径从 current working directory 开始；

- `openat()` 一类接口允许把“起点目录”显式变成一个目录 `fd`，从而减少对全局 `cwd` 的依赖并避免部分 `pathname race`。

4.4 硬链接：多个名字，同一个对象

$$\text{name}_1 \dashrightarrow I_X \dashleftarrow \text{name}_2.$$

核心结论：

- 两个名字地位对等；
- 创建硬链接增加对象的 `link count`；
- 删除一个目录项只删除一条命名边；
- 通常不能跨文件系统，因为 `inode number`/对象身份属于具体文件系统；
- 普通用户通常不能随意为目录建立硬链接，以免命名空间产生复杂环路。

4.5 符号链接：一个独立对象，内容解释为路径

符号链接本身有自己的文件对象/元数据，其内容语义上是目标 `pathname`。路径解析遇到 `symlink` 时，会根据规则继续解析目标路径，因此：

- 可跨文件系统；
- 可指向目录；
- 目标删除后 `symlink` 仍存在，但可能变成 `dangling link`；
- 它增加的是“路径解释层”，不是目标 `inode` 的 `link count`。

相对符号链接保存的是路径字符串，解析起点通常是链接所在目录；绝对符号链接从根开始。删除目标只会留下 `dangling link`，不会自动删除链接对象。硬链接与符号链接的判断顺序应固定为：是否共享同一 `inode`、是否增加目标 `link count`、能否跨文件系统、目标删除后是否仍可沿链接访问。

5 第 3 章：VFS 与 Mount——把多个文件系统拼成一个命名空间

5.1 为什么需要 VFS

如果每种文件系统都让应用调用不同 API，那么 `open/read/write/stat` 无法成为统一接口。VFS 的作用是：

Common filesystem API $\rightarrow$ polymorphic filesystem operations

它既向用户空间提供统一文件接口，也在内核中允许 `ext4`、`tmpfs`、`NFS` 等不同实现共存。

5.2 VFS 四类核心对象的职责

对象	回答的问题	稳定心智模型
superblock	这是哪个已挂载文件系统实例?	文件系统级状态/操作集合
inode	这个文件系统对象是谁?	元数据、权限、对象级操作
dentry	这个名字组件当前解析到谁?	内存 pathname cache/object binding
file	这一次打开实例怎样访问对象?	offset、status flags、访问操作

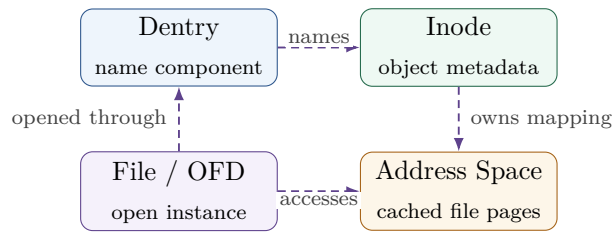


图 2: VFS 四对象：图中虚线统一表示“引用/关联”，不与动态控制流混淆。

5.3 Mount：路径经过挂载点时切换文件系统实例

挂载不是“把磁盘内容复制进目录”，而是让 pathname walk 到达某个 mount point 时，切换到另一个文件系统实例的根对象。因此：

one namespace tree ≠ one physical filesystem

同一路径树可以跨越多个 superblock/文件系统实例。

考试范围边界

408 常见范围包括文件元数据与 inode、文件操作、文件保护、逻辑/物理结构、目录、硬/软链接、文件系统全局布局、空闲空间管理、VFS 与挂载。Journaling、extents、dcache/XArray 用于深化机制或说明工程边界，不替代教材模型。

6 Part II：进程怎样引用已经找到的文件

7 第 4 章：fd、打开文件描述与 inode

7.1 为什么不能让 fd 直接等于 inode

如果 fd 直接指向 inode, 那么同一文件被独立打开两次时, 两个访问者如何拥有独立 offset? O_APPEND 等“打开实例状态”又放哪里?

于是必须增加一层：

fd → 打开文件描述（OFD） → inode

POSIX 语义中, 每次成功 open() 创建一个新的打开文件描述 (open file description, OFD); 文件 offset 和 file status flags 属于它, 而 fd 只是进程内的句柄。

7.2 三种最重要的共享关系

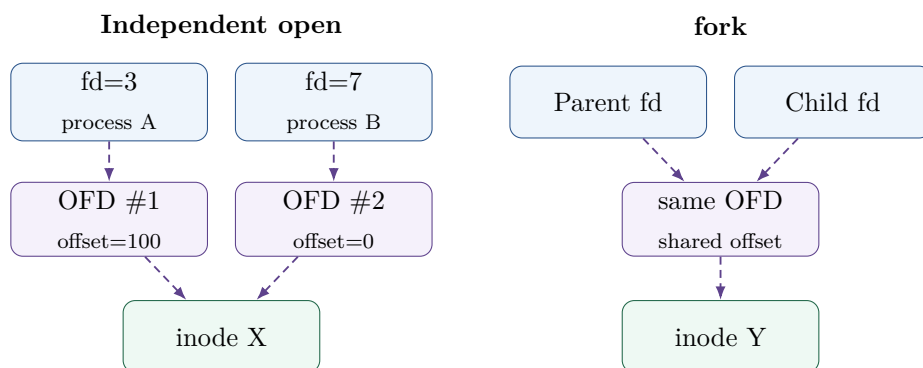


图 3: 独立 open 产生独立 OFD; fork 后对应 fd 引用同一 OFD, 因此共享 file offset。

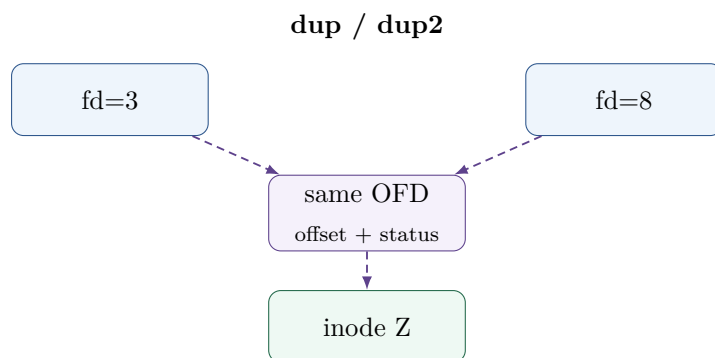


图 4: dup 不会再次 open, 而是增加一个 fd 句柄指向同一个 OFD。

7.3 一个稳定的题目判断法

遇到“offset 是否共享”时, 不要背父子/进程关系, 直接问:

Do these descriptors refer to the same OFD?

- 独立 open(): 通常不同 OFD, offset 独立;
- dup/dup2(): 同一 OFD, offset 共享;
- fork(): 子进程 fd 与父进程对应 fd 指向同一 OFD, offset 共享;
- exec(): 不是创建新进程; 未被 close-on-exec 关闭的 fd 可继续存在。

不要把“系统级打开文件表”想成“每个文件只有一个表项”

一个 inode 可以同时被多个 OFD 引用。文件对象身份和打开实例状态必须分开, 否则无法解释“同一文件独立打开两次 offset 不共享”。

8 第 5 章：文件生命周期——名字引用与打开引用是两套生命线

8.1 unlink 删除的首先是一条命名边

POSIX unlink() 的稳定语义是：删除 directory entry/link 并递减 link count。若最后一个 link 已删除但仍有打开引用, 文件内容的最终回收要推迟到这些引用关闭。

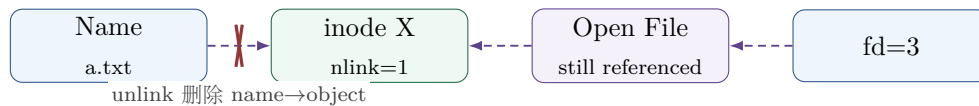


图 5: unlink while open: 名字可以消失, 而已经建立的打开引用继续访问同一对象。

8.2 双生命周期模型

- **Name lifetime:** 有多少持久目录链接仍指向对象? (经典 UNIX 用 link count 表达)
- **打开引用生命周期:** 运行时是否仍有 OFD 或其他内核引用使对象继续存活?

可以直接使用的结论是:

no persistent names + no live references $\Rightarrow$ eligible for final reclamation

Linux 实现边界

不要把教材中的“打开计数”机械等同于 Linux `inode.i_count`。`struct file`、inode cache、dentry cache 都有各自引用/回收机制。408 做题时可以用“链接引用 + 打开引用”两类计数理解生命周期; 工程实现则更细。

8.3 create、link、unlink、rename 分别改什么

操作	命名空间变化	对象/元数据变化
create	新增 name $\rightarrow$ new object	分配/初始化对象元数据, 通常 link count 从 1 开始
link	新增 second name $\rightarrow$ same object	link count 增加
unlink	删除一条 name $\rightarrow$ object	link count 减少; 是否最终回收看剩余引用
rename	把名字关系从旧位置移动/替换到新位置	通常不改变文件内容本身, 重点是目录元数据的一致更新

9 第 6 章：文件保护——谁可以对哪个对象做什么

文件保护的统一问题是:

Subject $\times$ Object $\times$ Operation $\rightarrow \{allow, deny\}$

408 常见机制包括访问权限, 以及所有者、所属组、其他用户的读写执行位。目录的读、写、执行权限与普通文件语义不同: 目录执行权限表示允许搜索或穿越, 写权限与创建、删除目录项相关。

保护题三问

1. 当前 subject 是谁 (用户/进程/权限域)?
2. 当前 object 是普通文件还是目录?
3. 请求 operation 是读取内容、修改内容、创建名字、删除名字还是穿越目录?

不要只看到“`rwX`”就机械对应同一含义。

10 Part III：文件中的字节怎样落到块上

11 第 7 章：从文件偏移到逻辑块

设文件系统块大小为 B ，字节偏移为 x ：

$$LBN = \left\lfloor \frac{x}{B} \right\rfloor, \quad \text{offset} = x \bmod B.$$

这一步只把文件内部字节位置转换成文件内部逻辑块号，还没有决定它在磁盘哪个位置。

三个“地址”不要混

- File offset：文件内的字节序号；
- LBN：文件自己的逻辑块号；
- filesystem block / device location：底层存储位置。

它们对应的层次不同，不能把 offset/B 的结果直接叫“磁盘块号”。

12 第 8 章：文件物理结构——三种经典方案从矛盾中产生

12.1 连续分配：数组模型

若文件占据连续块区间：

$$PBN = \text{start} + LBN.$$

优点：随机与顺序访问都直接，局部性好。**代价：**增长困难、外部碎片、预估大小困难。

12.2 链接分配：链表模型

每个数据块保存“下一块”的位置：

$$B_0 \rightarrow B_7 \rightarrow B_{19} \rightarrow \dots$$

优点：增长灵活、没有经典外部碎片问题。**代价：**隐式链随机访问差，指针损坏影响链后部。

FAT：把链从数据块搬进集中表 FAT 把“下一簇号”集中存储，使链遍历可在内存 FAT 中进行；因此它仍是 linked allocation 思想，但显著改善了定位开销（在 FAT 已在内存的题设下）。

12.3 索引分配：映射表模型

把数据块地址集中放进索引结构：

$$LBN \xrightarrow{\text{index}} PBN.$$

它和页表有明显同构：都通过增加一层 indirection，换来离散布局、随机定位和灵活增长。

VM 与 File System 的共同设计语言

$$\text{VPN} \rightarrow \text{Page Table} \rightarrow \text{PFN}$$

和

$$\text{LBN} \rightarrow \text{File Index} \rightarrow \text{Storage Block}$$

两条路径都采用“逻辑编号 → 映射结构 → 物理位置”。增加映射表后，系统可以移动、共享、稀疏分配或扩展底层空间，而不改变上层使用的逻辑编号。

13 第 9 章：inode 多级索引——最大文件与 I/O 次数计算

13.1 经典 UNIX-style mixed indexing

设一个 inode 中有：

- d 个直接指针；
- s 个一级间接指针；
- q 个二级间接指针；
- t 个三级间接指针。

文件系统块大小为 B ，块地址项大小为 a ，则一个索引块可容纳：

$$K = \left\lfloor \frac{B}{a} \right\rfloor$$

个地址项。

最大可寻址数据块数：

$$N_{max} = d + sK + qK^2 + tK^3$$

最大文件大小：

$$Size_{max} = N_{max} \cdot B$$

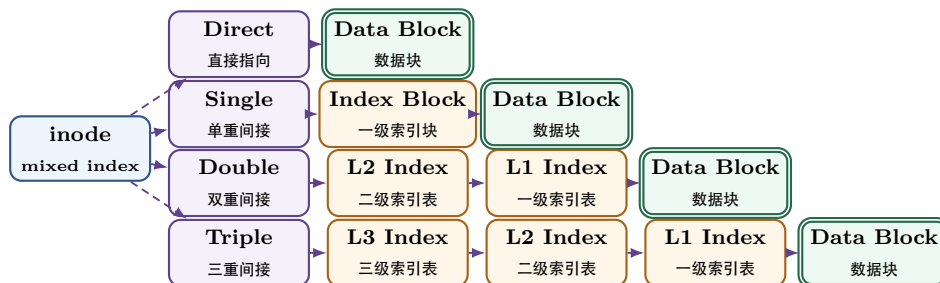


图 6: 经典 inode 混合索引。每增加一级间接，就多一次索引块解析机会与空间层级。

13.2 I/O 次数一定先写缓存假设

典型题设“inode 已在内存、相关索引块均未缓存”：

- 直接块：读数据需要 1 次块 I/O；

- 一级间接：索引块 + 数据块，共 2 次；
- 二级间接：两级索引 + 数据，共 3 次；
- 三级间接：三级索引 + 数据，共 4 次。

但如果索引块已缓存，次数会下降。因此任何 I/O 计算都先写：**inode 是否在内存？索引块是否缓存？目录块是否缓存？页缓存是否命中？**

现代 ext4: extent 不是 “12+1+1+1” 的简单延长

经典 direct/single/double/triple mixed-indexing 机制非常适合 408 中的索引计算；现代 ext4 可使用 extent tree，把“连续逻辑范围 → 连续物理范围”作为映射单位。Linux ext4 文档明确区分 inode 的 extent 格式等结构。工程边界用于理解“现代文件系统为何减少大文件映射元数据”，但不替代题设采用的经典 mixed-indexing 机制。

14 Part IV：空间从哪里来——Global Layout 与 Free Space

15 第 10 章：文件系统全局 Layout

15.1 为什么必须有全局账本

文件系统不能只存“每个文件的 inode”。它还必须知道：

- 文件系统总大小与块大小；
- 哪些 inode/数据块空闲；
- inode table 在哪里；
- 根对象与挂载状态；
- 日志、特性标志、校验信息等。

因此出现 superblock、bitmap、inode table、data region 等全局结构。

结构	解决的问题	典型信息
Superblock	这个文件系统实例整体是什么？	block size、总量、特性、状态等
Inode bitmap	哪些 inode identity 可分配？	每个 inode 的 free/used 状态
Block bitmap	哪些 data/fs blocks 可分配？	每块 free/used 状态
Inode table	每个对象元数据持久放在哪里？	inode records
Data region	文件/目录的内容在哪里？	data blocks / index / extents
Journal（扩展）	多结构更新如何恢复？	transaction records / commit

ext4 block groups

现代 ext4 会把磁盘划成 block groups，并在组内组织 bitmap、inode table 与数据块，以提高局部性并减少碎片/寻道成本。它说明一个更普遍的设计原则：**空间分配不仅追求“能分到”，还追求 metadata 与 data 的局部性。**

16 第 11 章：外存空闲空间管理

16.1 位图 Bitmap

用 1 bit 表示一个单位是否空闲。优点是结构紧凑，寻找连续空闲范围和统计较方便；代价是需要扫描/维护位图并保持一致性。

16.2 空闲链表 Free List

把空闲块串起来。结构简单，但想找一段连续空间或一次分配很多块时可能遍历成本高。

16.3 成组链接 Grouped Linking

把若干空闲块号作为一组集中保存，再用某个块连接下一组。经典 UNIX 教学模型用它降低频繁访问长链的成本。

16.4 计数法 Counting

当空闲块经常成片连续时，用：

$(start, count)$

记录一段空闲区。它体现一个普遍优化：不要逐块记录具有强连续性的状态，而要记录范围。

空闲空间问题不要只问“哪个方法快”

固定四问：元数据多大？找一个块快不快？找连续 k 块快不快？回收/合并是否方便？不同 workload 下不存在绝对最好方案。

位图题还要先确定“1 表示空闲还是已用”的题设约定，再按块号定位 bit；成组链接的组首保存若干空闲块号，最后一个入口指向下一组，组满/组空时才发生组间搬移。计数法适合连续空闲区，空闲链表适合简单回收，位图适合快速判断单块状态；选择取决于查询对象是单块还是连续范围。

17 Part V：文件操作怎样改变内核状态

18 第 12 章：一次 open() 的一生

18.1 open 的高层语义

open(path, flags) 不是“把文件内容读进来”，而是建立：

pathname $\rightarrow$ object $\rightarrow$ new OFD $\rightarrow$ fd

其中 open() 的 POSIX 语义包括创建新的 OFD，并让一个 fd 指向它；常规文件初始 offset 通常从文件开头开始（特定 flag 另行处理）。

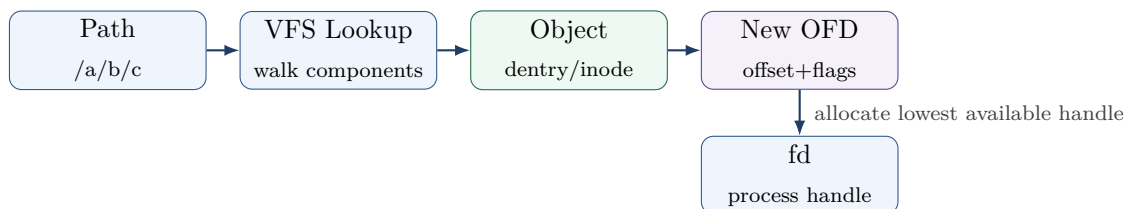


图 7: `open()` 的核心不是读数据，而是把“名字”转成“运行时访问关系”。

19 第 13 章：一次 `read(fd, buf, n)` 的文件系统路径

19.1 先定位打开实例，再定位文件位置

$$fd \longrightarrow OFD \longrightarrow \{offset, flags, object\}.$$

随后根据 `offset` 计算文件页/逻辑块位置。

19.2 现代 buffered read：先问页缓存

普通 buffered file I/O 中，更合理的逻辑是：

$$(file, page\ index) \longrightarrow Page\ Cache?$$

而不是先无条件算 PBN 再查缓存。若命中，直接从缓存页向用户缓冲提供数据；若未命中，具体文件系统才需要建立文件偏移到存储位置的映射并向 I/O 子系统提交读取。

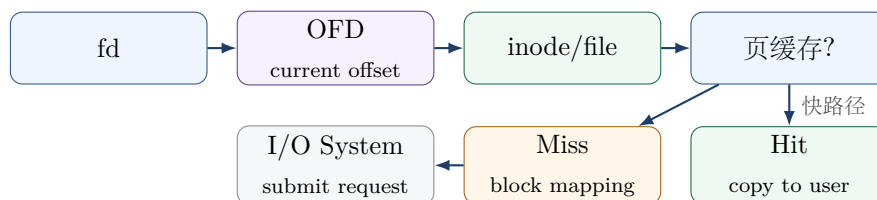


图 8: `read()` 的文件系统边界：到 I/O request 为止；DMA/interrupt/wakeup 已由 I/O 专题负责。

19.3 跨块读取

如果读取范围跨多个文件页/块，不能只计算起始 LBN：

$$LBN_{start} = \left\lfloor \frac{off}{B} \right\rfloor, \quad LBN_{end} = \left\lfloor \frac{off + n - 1}{B} \right\rfloor.$$

实际读取字节数还受到 EOF 限制。

文件操作状态变化

create 建立对象并把名字接入父目录；**open** 建立新的 OFD 与 fd；**dup/fork** 可共享同一 OFD 及其 offset；**close** 只释放一个句柄引用；**read/write** 推进共享 offset（除非使用显式位置接口）；**lseek** 改变 offset 而不读写数据；**truncate** 改变文件大小并可能释放数据块；**unlink** 删除命名边，仍被打开的对象直到最后一个引用消失才回收。做题时逐项标记 namespace、inode、OFD、fd、offset、data blocks 哪一层改变。

20 第 14 章：一次 `write(fd, buf, n)` 的文件系统路径

20.1 Buffered write 的核心状态变化

常见普通写路径：

User Data $\rightarrow$ 页缓存 $\rightarrow$ Dirty

并更新文件 `offset`、`size`/`mtime` 等相关内存元数据。系统调用返回与“数据已经稳定落盘”不是同一时间点。

与 I/O 专题接口

本册只追到：**dirty cached pages + block mapping + writeback request**。请求提交以后，设备队列、driver、DMA、completion interrupt 等交给 I/O 专题。这样两册之间只有接口，不重复整条设备路径。

20.2 写入可能同时修改哪些状态

- 文件数据页；
- file size（若扩展）；
- `mtime`/`ctime` 等元数据；
- block allocation metadata（若需要新块）；
- block mapping/index/extent；
- OFD 的 `offset`。

正因为一次高层 `write` 可能对应多份持久结构更新，后面才会产生 crash consistency 问题。

21 第 15 章：create、close、unlink、rename 的状态机

21.1 create：先创造身份，再绑定名字

以经典 inode 模型看，新建文件至少需要：

1. 找到父目录；
2. 分配对象身份/inode；
3. 初始化 metadata；
4. 在父目录中建立 `name  $\rightarrow$  inode` 映射；
5. 若同时打开，再创建 OFD 与 `fd`。

创建空文件时不必立刻拥有数据块；实际数据块可在后续写入时按需分配。你原有材料把“inode bitmap、inode table、parent directory”识别为核心持久结构，这是理解 crash consistency 的好入口。

21.2 close：释放一个 handle，不等于删除文件

`close(fd)` 首先终止当前 `fd` 对 OFD 的引用。只有当相应运行时引用都消失，OFD 才可回收；而持久文件是否存在仍由命名空间链接等状态决定。

21.3 rename：名字变化不等于内容复制

高层语义重点是对 namespace 的原子式名称变更/替换要求，而不是“把全部数据重新写一遍”。这也是为什么 `rename` 常被应用用作构造持久更新协议的一部分，但具体持久化保证仍要结合文件系统与同步调用分析。

22 Part VI：为什么“写完”以后文件系统仍可能坏

23 第 16 章：Crash Consistency——持久状态机的核心问题

23.1 高层一次操作，底层多次写

假设 `append/create` 需要更新：

$$W_1 = \text{bitmap}, \quad W_2 = \text{inode}, \quad W_3 = \text{directory/data}.$$

设备只能在时间上逐步持久化它们，崩溃可以发生在任意两步之间。因此我们要的是：

after reboot, on-disk state must correspond to a legal filesystem state

而不是假设“一次系统调用天然就是一次原子磁盘事务”。

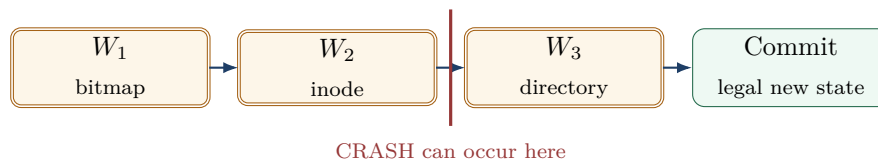


图 9: Crash consistency 的数学对象不是单个写，而是“多写序列 + 任意崩溃点”。

23.2 崩溃分析五步法

Crash Point Analysis

1. 列出高层操作会修改的所有持久对象；
2. 写出允许/要求的持久化顺序；
3. 标出 commit point 或“完成”的判据；
4. 在每两次持久写之间插入 crash；
5. 对每个 crash state 问：哪些结构互相矛盾？重启后是 ignore、redo、undo 还是 repair？

24 第 17 章：fsck 与 Journaling

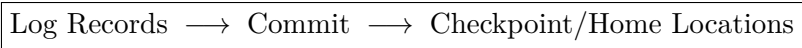
24.1 fsck：允许不一致，重启后扫描修复

早期思路可以概括为：正常运行不维护完整事务日志，crash 后遍历 inode、bitmap、目录等结构，检查引用数、空间使用与对象关系是否自洽，再修复。

优势：运行时机制简单。代价：大文件系统全盘检查慢，而且“修复到一致”不一定等价于恢复用户最期望的最近状态。

24.2 Journaling：先记录一笔可重放的事务

经典 write-ahead logging 思路：



只有看到有效 commit 的事务才视为已提交；重启时可以根据日志 replay 已提交但尚未 checkpoint 完成的修改。

工程边界：ext4/JBD2

ext4 使用 jbd2 journal 保护文件系统免于元数据更新中途崩溃造成结构不一致。官方文档说明：事务完整写入并有 commit record 后，可在恢复时 replay；默认 data=ordered 模式主要 journal metadata，并要求相关 data 在 metadata commit 前按规则写出。这是一种现代实现实例，不应替代 408 对 journaling/WAL 一般机制的理解。

24.3 Consistency 与 Durability 不完全是一件事

- **Consistency**：磁盘上的结构彼此不矛盾、符合文件系统不变量；
- **Durability/Freshness**：应用刚写的数据到底已经持久到什么程度。

文件系统可以在 crash 后结构一致，但丢失最近尚未达到持久边界的数据。因此 write() 返回、后台 writeback、journal commit、fsync() 完成、设备 cache flush 是不同层次的时间点。

25 第 18 章：概念边界

对象 A	≠	对象 B	真正区别与题目信号	混淆后果
filename	≠	inode / FCB	名字属于目录命名关系；inode/FCB 表示持久对象的身份与元数据。	把 rename/link/unlink 错写成直接修改对象身份。
directory entry	≠	dentry	前者可指持久目录记录；Linux dentry 是 VFS 的内存命名缓存对象。	把教材磁盘结构与工程内存对象混成一层。
fd	≠	inode	fd 是进程句柄，通常先引用打开文件描述；inode 是持久文件对象。	无法解释同一 inode 上多个独立 open 的不同 offset。
OFD	≠	inode	OFD 保存一次打开实例的 offset/flags 等运行时状态；多个 OFD 可以共享同一 inode。	把独立 open、dup、fork 的共享关系判错。

对象 A	≠	对象 B	真正区别与题目信号	混淆后果
link count	≠	open reference	前者统计持久命名边；后者反映运行时打开引用。	误以为 <i>nlink</i> = 0 时数据必然立即释放。
file offset	≠	PBN	offset 先定位 file page/LBN，再经文件映射结构得到存储位置。	把逻辑文件位置直接当成设备块号。
页缓存	≠	block mapping	页缓存按文件对象与页索引保存内容副本；block mapping 在 miss/后端访问时解析存储位置。	无条件先求 PBN，错误增加底层 I/O。
FS block	≠	sector / atomic write unit	文件系统分配单位、设备扇区和设备原子写保证不是同一层概念。	错判空间利用率、I/O 粒度或 crash 原子性。
write() 返回	≠	durable	接口返回、后台 writeback、journal commit、fsync() 和设备稳定介质属于不同时间边界。	把“内核已接受”误写成“断电后必然存在”。
经典 mixed inode	≠	ext4 extent	前者是 408 的经典索引模型；现代 ext4 常使用 extent tree 表示连续范围。	用现代实现细节替换题设经典模型，或反过来把经典结构冒充现代事实。

26 第 19 章：最小调用接口

遇到文件系统问题时，先判当前对象属于 **Name、Object、Open State、Data Mapping 还是 Persistent Protocol**。Canonical 负责解释这些对象怎样存在、引用和变化；固定十四问、路径/打开实例状态追踪与 ‘unlink’ 变式进入 路径与文件对象.md，mixed indexing、字节定位与 I/O 次数进入 文件索引与 IO 计算.md，crash point、fsck/journaling 与持久边界进入 崩溃一致性.md。

第一观察点始终是：**当前操作改变的是哪一层状态？**若问题继续进入设备 request、DMA、interrupt 或 block/wakeup，则停止在本册展开并转交 OS-05 / OS-I01。

三条核心结论

文件系统最值得记住的不是某个 inode 字段，而是三句话：

1. **Naming is mapping**: 名字只是到对象的引用，不等于对象本身。

2. **Open is state**: 打开以后必须有独立的运行时访问状态，因此 fd、OFD、inode 必须分层。

3. **Persistence is protocol**: 高层一次操作会变成多个持久写，正确性来自写序、提交与恢复协议，而不是“磁盘自己不会坏”。

Name → Object → Open State → Data Mapping → Persistent Protocol

27 附录 A：考试模型 / 机制 / 工程边界

主题	考试模型	机制深化	工程边界
文件抽象	文件、元数据、inode/FCB、操作、保护	object identity、metadata/data split	VFS inode operations
目录	树形目录、路径、操作、硬/软链接	pathname state machine	dcache、negative dentry、openat
打开文件	open/close/read/write、共享	fd→OFD→inode	Linux struct file/files_struct
物理结构	contiguous/linked/indexed	mixed indexing、I/O path	extents/delayed allocation
空间管理	bitmap/free list/grouping 等	locality/range allocation	ext4 block groups
文件系统	layout、VFS、mounting	namespace vs filesystem instance	VFS core objects
持久性	理解写回与一致性思想	fsck/journaling/crash points	JBD2 / ext4 data modes

28 附录 B：参考资料与工程边界

正文以 408 教材模型为主，以下公开资料用于核对术语和工程实现边界：

- Linux Kernel Documentation, *Overview of the Linux Virtual File System*: <https://docs.kernel.org/filesystems/vfs.html>
- Linux Kernel Documentation, *ext4 Data Structures and Algorithms*: <https://docs.kernel.org/filesystems/ext4/>
- Linux Kernel Documentation, *ext4 Journal (jbd2)*: <https://docs.kernel.org/6.17/filesystems/ext4/journal.html>
- POSIX.1-2024, *open()*, *dup()*, *fork()*: <https://pubs.opengroup.org/onlinepubs/9799919799/>
- OSTEP, *File System Implementation* 与 *Crash Consistency: FSCK and Journaling*: <https://pages.cs.wisc.edu/~remzi/OSTEP/>